

Relazione - Versione 3: Generic Parser

Introduzione

La Versione 3 introduce il Generic Parser.

Nella Versione 2, il Parser funzionava, ma era ancora rigido.

Utilizzava condizioni scritte direttamente nel codice per specifiche combinazioni di parametri.

Per esempio:

```
if parameters == ["name"]:  
    ...  
  
if parameters == ["a", "b"]:  
    ...
```

Questo approccio funziona con un numero ridotto di strumenti, ma non è facilmente scalabile.

Se vengono aggiunti nuovi strumenti o nuovi parametri, il Parser si riempirebbe rapidamente di istruzioni if.

La Versione 3 risolve questo problema rendendo il Parser più generico.

Obiettivo della Versione 3

L'obiettivo principale della Versione 3 è eliminare la logica di parsing rigida.

Il Parser non deve più dipendere da configurazioni specifiche degli strumenti.

Deve invece:

1. leggere i parametri richiesti dallo strumento selezionato;
2. trovare l'estrattore corretto per ciascun parametro;
3. estrarre ogni valore;
4. restituire il dizionario finale degli argomenti.

L'idea chiave è:

Il Parser non deve conoscere ogni strumento nel dettaglio.
Deve sapere come estrarre i parametri.

Cosa cambia rispetto alla Versione 2

Nella Versione 2, il Parser era basato su condizioni specifiche.

Controllava l'elenco esatto dei parametri e decideva cosa fare.

Nella Versione 3, il Parser diventa basato sui parametri.

Il nuovo flusso è:

```
strumento selezionato
↓
parametri richiesti
↓
estrattori dei parametri
↓
argomenti
```

Questo rende il Parser più flessibile e più facile da estendere.

Contesto di parsing

La Versione 3 introduce il concetto di contesto di parsing.

Il contesto di parsing raccoglie informazioni utili dalla richiesta dell'utente.

Esempio di richiesta:

```
Quanto fa 2 + 3?
```

Il contesto può essere:

```
{
  "numbers": [2, 3],
  "name": None
}
```

Questo contesto viene poi riutilizzato dagli estrattori dei parametri.

Il vantaggio è che la richiesta viene analizzata una sola volta e le informazioni estratte vengono condivise all'interno del Parser.

Estrattori dei parametri

Gli estrattori dei parametri sono piccole funzioni incaricate di estrarre uno specifico valore.

Esempi:

"name" → estrae il nome dell'utente
"a" → estrae il primo numero
"b" → estrae il secondo numero

La mappatura può apparire così:

```
parameter_extractors = {  
  "name": name_extractor,  
  "a": first_number_extractor,  
  "b": second_number_extractor  
}
```

Ciò consente al Parser di funzionare in modo generico.

Non deve conoscere la logica completa di ogni strumento.

Deve soltanto sapere quale estrattore corrisponde a ciascun parametro.

Flusso del Generic Parser

Esempio di richiesta:

Quanto fa 2 + 3?

Il Router seleziona:

"addition"

I metadati dello strumento indicano che l'addizione richiede:

["a", "b"]

Il Parser costruisce il contesto:

```
{  
  "numbers": [2, 3],
```

```
"name": None
}
```

Quindi estrae ciascun parametro:

```
"a" → primo numero → 2
"b" → secondo numero → 3
```

Gli argomenti finali analizzati sono:

```
{
  "a": 2,
  "b": 3
}
```

Architettura completa nella Versione 3

Il flusso della Versione 3 è:

```
Richiesta dell'utente
↓
Router
↓
Generic Parser
↓
Executor
↓
Memoria
```

Le responsabilità principali ora sono più chiare:

```
Router = seleziona lo strumento
Parser = estrae gli argomenti
Executor = convalida ed esegue
Agent = coordina
Memory = memorizza lo stato
```

Questa è già un'architettura molto più solida rispetto alla Versione 1.

Perché il Generic Parser è importante

Il Generic Parser è importante perché rende il sistema più facile da estendere.

Se uno strumento futuro richiede un nuovo parametro, il Parser non deve essere riscritto aggiungendo altre condizioni rigide.

Per esempio, se uno strumento futuro richiede:

```
city
```

possiamo aggiungere:

```
"city": city_extractor
```

In questo modo il Parser rimane pulito.

Il sistema diventa più scalabile, perché aggiungere un parametro significa aggiungere un estrattore, non riscrivere il Parser.

Test

I test verificano che il nuovo Parser non alteri il comportamento precedente.

Per esempio:

```
agent("Quanto fa 2 + 3?")
```

deve continuare a restituire:

```
5
```

Un altro test:

```
agent("Ciao, quanto fa 2 + 3?")
```

deve continuare a selezionare:

```
"addition"
```

E:

```
agent("Ciao, mi chiamo luca.")
```

deve continuare a restituire:

Ciao Luca, piacere di conoscerti!

Questi test confermano che il Parser è più generico, mentre il comportamento dell'agente rimane corretto.

Cosa insegna la Versione 3

La Versione 3 insegna un importante principio architetture:

i componenti generici sono più scalabili dei componenti rigidi

Il Parser non dipende più da specifici casi scritti direttamente nel codice.

Legge i metadati e utilizza gli estrattori dei parametri.

Questo rende l'agente:

- più pulito;
 - più facile da estendere;
 - più facile da leggere;
 - più facile da modificare;
 - più vicino a un'architettura professionale.
-

Limitazioni

La Versione 3 presenta ancora alcune limitazioni:

- il Router è ancora basato su regole;
- il Parser utilizza ancora regole di estrazione manuali;
- la memoria è ancora una semplice lista;
- non esiste un Planner;
- non esiste un Tool Registry;
- non è presente alcun LLM;
- l'agente esegue ancora una sola azione alla volta.

Queste limitazioni sono normali.

Verranno affrontate nelle versioni successive.

Conclusione

La Versione 3 è un passo importante verso un agente più scalabile.

Il Parser diventa generico e non dipende più da istruzioni if rigide per ogni configurazione degli strumenti.

L'agente ora presenta un flusso più pulito:

```
Router
↓
Generic Parser
↓
Executor
↓
Memoria
```

Il passo successivo naturale è introdurre un Planner, affinché l'agente possa creare un piccolo piano di esecuzione prima di avviare uno strumento.